1. INTRODUCTION

1.1. About the Project

Cardiac disease, also called coronary artery disease, is a condition that affects the heart. Cardiac disease is a leading cause of death worldwide. Physicians generally make decisions by evaluating current test results of the patients. Previous decisions taken by other patients with the same conditions are also examined. Diagnosing cardiac disease requires experience and highly skilled physicians. Cardiac disease was projected to become a leading cause of death by 2020. Cardiac disease diagnosis is an important yet complicated task.

Today, many hospitals collect patient data to manage the health care of patients. This information is in different formats like numbers, charts, text, and images. This database contains rich information but is poorly used for clinical decision-making. The automation or decision support system would be extremely advantageous.

Data mining can be used to automatically infer diagnostic rules and help specialists make the diagnosis process more reliable. The purpose of predictions in data mining is to discover trends in patient data to improve their health care. Knowledge discovery in databases is applied to extract useful patterns from medical data sets using various data mining techniques. Data mining has shown good results in the prediction of cardiac disease and is widely applied for this purpose.

Due to the shortage of doctors and experts in the medical field to predict cardiac disease, and because of neglecting the patient's symptoms, data mining has emerged as an analysis tool. Several data mining techniques are used by researchers to help health care professionals predict cardiac disease.

Random Forest is an ensemble and highly accurate learning algorithm, suitable for medical applications. Random Forest is an ensemble classifier that combines bagging and random selection of features. It can handle data without preprocessing. The Random Forest algorithm has been used in prediction and probability estimation. Random Forest consists of many decision trees and outputs the class, which is the mode of individual trees classes. It is one of the most accurate classifiers. It produces highly accurate classifications for many datasets, especially cardiac disease datasets.

2. SYSTEM SPECIFICATION

2.1 Hardware Requirements

System	Intel Core2 due Processor 2 GHz Speed
Hard Disk	500 GB
Monitor	15’’ VGA color Monitor
Mouse	Optical Mouse
Keyboard	Standard Keyboard
RAM	1GB

2.2 Software Requierements

Operating System	Windows 10.
Language	PYTHON
GUI Tool	Tkinter
Editor	Python IDE

2.2.1. Python

Python is a high-level, interpreted and general-purpose dynamic programming language that focuses on code readability. The syntax in Python helps the programmers to do coding in fewer steps as compared to Java or C. Python is widely used in bigger organizations because of its multiple programming paradigms. They usually involve imperative and object-oriented functional programming. It has a comprehensive and large standard library that has automatic memory management and dynamic features.

The software development companies prefer Python language because of its versatile features and fewer programming codes. Nearly 14% of the programmers use it on the operating systems like UNIX, Linux, Windows and Mac OS. Interactive

Advantages or Benefits of Python

The Python language has diversified application in the software development companies such as in gaming, web frameworks and applications, language development, prototyping, graphic design applications, etc. This provides the language a higher plethora over other programming languages used in the industry. Some of its advantages are:

Extensive Support Libraries

It provides large standard libraries that include the areas like string operations, Internet, web service tools, operating system interfaces and protocols. Most of the highly used programming tasks are already scripted into it that limits the length of the codes to be written in Python.

Presence of Third Party Modules

The Python Package Index (PyPI) contains numerous third-party modules that make Python capable of interacting with most of the other languages and platforms.

- **Integration Feature**

Python integrates the Enterprise Application Integration that makes it easy to develop Web services by invoking COM or COBRA components. It has powerful control capabilities as it calls directly through C, C++ or Java via Jython. Python also processes XML and other markup languages as it can run on all modern operating systems through same byte code.

- **Improved Programmer's Productivity**

The language has extensive support libraries and clean object-oriented designs that increase two to ten fold of programmer's productivity while using the languages like Java, VB, Perl, C, C++ and C#.

- **Productivity**

With its strong process integration features, unit testing framework and enhanced control capabilities contribute towards the increased speed for most applications and productivity of applications. It is a great option for building scalable multi-protocol network applications.

- **Open Source and Community Development**

Python language is developed under an OSI-approved open source license, which makes it free to use and distribute, including for commercial purposes.

Further, its development is driven by the community which collaborates for its code through hosting conferences and mailing lists, and provides for its numerous modules.

- **Learning Ease and Support Available**

Python offers excellent readability and uncluttered simple-to-learn syntax which helps beginners to utilize this programming language. The code style guidelines, PEP 8, provide a set of rules to facilitate the formatting of code. Additionally, the wide base of users and active developers has resulted in a rich internet resource bank to encourage development and the continued adoption of the language.

- **User-friendly Data Structures**

Python has built-in list and dictionary data structures which can be used to construct fast runtime data structures. Further, Python also provides the option of dynamic high-level data typing which reduces the length of support code that is needed.

- **Simple and easy to Learn**

1. The syntaxes of python language are very simple. Anybody can remember the python language syntaxes, rules and regulations very easily.
2. The elegant syntaxes of the python language makes the people to learn python in easiest manner.

3. With out having any other programming languages knowledge, we can learn python directly.
4. The simple and powerful syntax of python language makes the programmers to express their business logic is less lines of code.

- **Platform Independent**
  1. Like Java programs, Python programs are also platform independent.
  2. Once we write a Python program, it can run on any platform without rewriting once again.
  3. Python uses PVM to convert python code to machine understandable code.

- **High Level Language**
  1. Python is a high level language.
  2. While developing python applications, developers no need to bother about memory management.
  3. Dynamically Typed Language
  4. Python is a dynamically typed language. I mean, in python there is no need to declare type of a variable.
  5. Whenever we assign a value to the variable, based on the value the type will be allocated automatically.

- **Python supports POP & OOP**
  1. Python language supports both Procedural Oriented Programming and Object Oriented Programming features.
  2. We can implement OOPs features like Encapsulation, polymorphic, inheritance and abstraction in python programming language.

- **Python is interpreted Language**
  1. Like PHP, Python is also interpreted language.
  2. Python applications do not require explicit compilation so that compiler is not require for python software.
  3. Python interpreter is responsible for execution of python applications.

4. When ever we run a python application, the python interpreter checks for the syntax errors. If there is no syntax errors found in our code then the interpreter converts the code into intermediate code in the form of low level format and executes it.
5. The intermediate code of the python applications is known as byte code.
6. The extension of the byte code file is .pye (Python Compile Code).

- **Python is Embeddable**
  1. We can embed the python code into other languages such as C,C++, Java and Etc..
  2. In order to provide the scripting capabilities to other languages we can use the python code in it.
- **Develop GUI & Web Application**
  1. We can develop the GUI based applications using Python Languages.
  2. We can also develop the Web applications using Python Language.

Applications of Python

- GUI based desktop applications
  - Image processing and graphic design applications
  - Scientific and computational applications
  - Games
- Web frameworks and web applications
- Enterprise and business applications
- Operating systems
- Language development
- Prototyping

2.2.2. Tools and Technology Used

Flask

The Flask framework is actually a glue, a very nice one, that sticks together the amazing Werkzeug and Jinja2 frameworks, responsible for answering requests and presenting the output (HTML, maybe). In the MVC architecture, also known as Model-View-Controller, Flask covers C and V. Flask does not provide you with an integrated model layer out-of-the-box as that is not actually needed for a web application. If you do need to work with a

database, just pick your database solution from the many available and create your own model layer, which is not hard, and be happy. The concept of a micro-framework, with good intentions and made just for Flask, is all about giving you the smallest (but also the most useful) feature set you need, and one that won't get in the way.

Just what are the features that must be in the framework

- A development server and debugger
- Unicode support
- WSGI compliance
- A unit-test client
- URL routing
- Request dispatching
- Secure cookies
- Sessions
- Jinja2 templates

Why is Flask a good web framework choice

Flask is considered more Pythonic than the Django web framework because in common situations the equivalent Flask web application is more explicit. Flask is also easy to get started with as a beginner because there is little boilerplate code for getting a simple app up and running.

How does Flask relate to the Pallets Projects

Flask was originally designed and developed by Armin Ronacher as an April Fool's Day joke in 2010. Despite the origin as a joke, the Flask framework became wildly popular as an alternative to Django projects with their monolithic structure and dependencies.

Flask's success created a lot of additional work in issue tickets and pull requests. Armin eventually created The Pallets Projects collection of open source code libraries after he had been managing Flask under his own GitHub account for several years. The Pallets Project now serves as the community-driven organization that handles Flask and other related Python libraries such as Lektor, Jinja and several others.

3. SYSTEM STUDY

3.1. Existing System

The major challenge in cardiac disease is its detection. Although instruments are available to predict cardiac disease, they are either expensive or inefficient in accurately calculating the likelihood of cardiac disease in humans. Early detection of cardiac diseases can significantly decrease the mortality rate and reduce overall complications. However, it is not feasible to monitor patients every day in all cases accurately, and 24-hour consultation by a doctor is unavailable, as it requires substantial expertise, time, and effort.

With the abundance of data available today, various machine learning algorithms can be employed to analyze this data for hidden patterns. These hidden patterns can then be utilized for health diagnosis in medicinal data.

Prediction using traditional methods and models involves various risk factors and consists of various measures of algorithms such as datasets, programs, and much more. High-risk and low-risk patient classification is done based on the tests conducted in groups. However, these models are only valuable in clinical situations and not in large industry sectors.

To include disease predictions in various health-related industries, we have used the concepts of machine learning and supervised learning methods to build the prediction system.

3.1.1 Drawbacks in Existing System

- Nowadays, in the health industry, there are various problems related to machines or devices that provide incorrect or unacceptable results. To avoid such outcomes and achieve accurate and desired results, improvements are necessary.
- The health industry is rich in information yet poor in knowledge. This is a vast sector with significant room for development and optimization

3.2 Proposed System

The proposed system predicts cardiac disease by exploring four classification algorithms and performing a performance analysis. The objective of this study is to effectively predict whether a patient suffers from cardiac disease.

In this system, health professionals enter input values from the patient's health report. This data is fed into the model, which predicts the probability of having cardiac disease. The Random Forest algorithm is used for its high accuracy in prediction.

The machine learning technique for cardiac disease prediction identifies significant features. The ML process begins with a data pre-processing phase, followed by:

- Feature selection based on data cleaning
- Classification modeling
- Performance evaluation

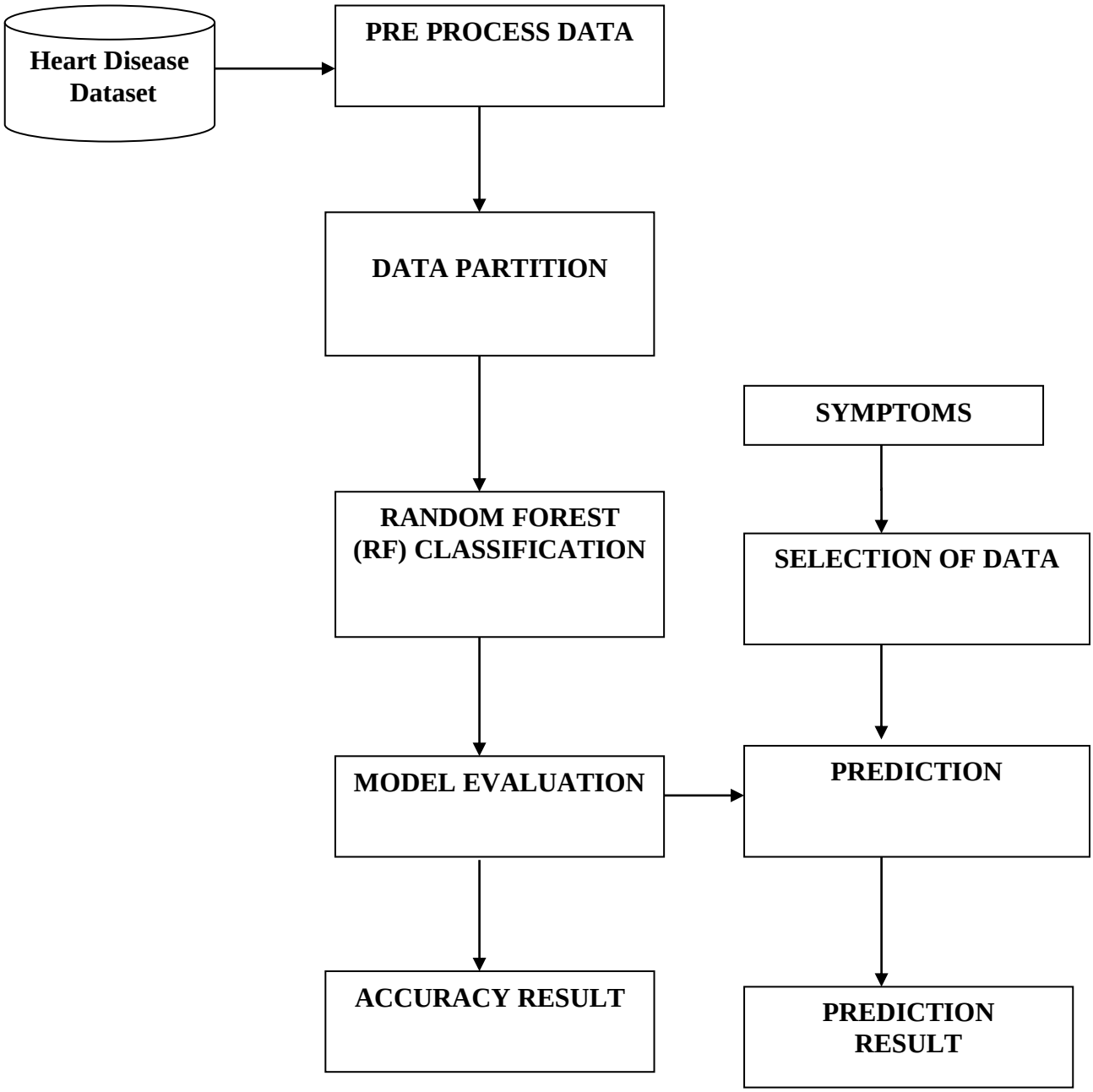
This results in improved accuracy for predictions.

3.2.1. Advantages of Proposed System

- This project predicts diseases based on the patient's symptoms and general information using datasets.
- By analyzing hospital datasets, the system achieves up to 80% accuracy in predictions, with ongoing improvements aimed at reaching 100% accuracy.
- Disease prediction helps identify patient conditions early, solving various problems and preventing adverse outcomes.
- The system provides robust security to prevent unauthorized access or modifications.
- Compared to other machine learning techniques, this system achieves higher accuracy.

4. SYSTEM DESIGN AND DEVELOPMENT

4.1. System Archieteure Diagram



4.2 Module Description

1. **Data Collection Module**: This module is responsible for gathering comprehensive heart health data from users. It includes a variety of inputs such as age, gender, blood pressure, cholesterol levels, and other relevant health metrics. The data collection is facilitated through a user-friendly interface on the Django web platform, ensuring ease of use and accessibility for individuals with varying levels of technical proficiency.

2. **Random Forest Model Module**: At the heart of the project is the Random Forest Model Module, which employs the Random Forest algorithm for classification purposes. This module is implemented using Python and is tasked with analyzing the preprocessed data to identify patterns and relationships indicative of heart disease risk. The Random Forest algorithm is chosen for its efficiency in handling complex datasets and its ability to provide high accuracy in prediction outcomes.

3. **Prediction and Analysis Module**: This module integrates closely with the Random Forest Model to interpret the algorithm's output and translate it into meaningful insights. It assesses the likelihood of heart disease based on the analyzed data and quantifies the risk level. The analysis considers the intricate interactions between various health indicators, providing a comprehensive evaluation of heart health.

4. **User Interface and Feedback Module**: Developed using Django, this module presents the final prediction and analysis results to the user through a clear and intuitive web interface. It displays the risk assessment in an understandable format, allowing users to grasp their heart health status quickly. Additionally, this module may offer personalized recommendations or advice based on the prediction results, guiding users on potential next steps or lifestyle changes.

5. **Data Preprocessing Module**: Once the data is collected, this module takes over to clean and prepare the data for analysis. It involves handling missing values, normalizing data ranges, and converting categorical data into a format suitable for machine learning models. The preprocessing steps are crucial for enhancing the model's performance by ensuring the input data is consistent and accurately reflects the underlying patterns related to heart health.

4.3 Dataset Description

This dataset is designed to facilitate the prediction of heart disease risk based on various health-related features. It includes a collection of attributes that reflect an individual's health status and lifestyle factors known to influence heart disease risk. Typically, the data will include both continuous and categorical variables that represent different health metrics and demographics.

The dataset for this project includes the following key features

1. Age

- **Type**: Continuous (Numerical)
- **Description**: The age of the individual. Age is a significant factor in heart disease risk, with older individuals generally having a higher risk.

2. Gender

- **Type**: Categorical (Binary)
- **Description**: The gender of the individual. It is generally encoded as 0 for female and 1 for male. Gender can influence heart disease risk, as men typically have a higher risk at younger ages compared to women.

3. Chest Pain Type (cp)

- **Type**: Categorical
- **Description**: This feature represents the type of chest pain experienced by the individual. It can be categorized into four types:
    - **Value 1**: Typical angina
    - **Value 2**: Atypical angina
    - **Value 3**: Non-anginal pain
    - **Value 4**: Asymptomatic

4. Resting Blood Pressure (trestbps)

- **Type**: Continuous (Numerical)
- **Description**: The resting blood pressure of the individual (in mm Hg). High blood pressure is a major risk factor for heart disease.

5. Serum Cholesterol (chol)

- **Type**: Continuous (Numerical)
- **Description**: The serum cholesterol level of the individual (in mg/dl). Elevated cholesterol levels are a known risk factor for heart disease.

6. Fasting Blood Sugar (fbs)

- **Type**: Categorical (Binary)
- **Description**: Indicates whether the individual's fasting blood sugar level is greater than 120 mg/dl. This is typically used as a diagnostic criterion for diabetes, which is related to heart disease risk.
    - **1**: True (greater than 120 mg/dl)
    - **0**: False (less than or equal to 120 mg/dl)

7. Resting Electrocardiographic Results (restecg)

- **Type**: Categorical
- **Description**: The resting electrocardiographic result of the individual. This is typically a measure of the electrical activity of the heart.
    - **Value 0**: Normal
    - **Value 1**: Having ST-T wave abnormality (possibly showing signs of ischemia)
    - **Value 2**: Showing probable or definite left ventricular hypertrophy

8. Maximum Heart Rate Achieved (thalach)

- **Type**: Continuous (Numerical)
- **Description**: The maximum heart rate achieved by the individual during exercise. A lower heart rate during exercise may indicate cardiovascular problems.

9. Exercise Induced Angina (exang)

- **Type**: Categorical (Binary)
- **Description**: Indicates whether the individual experienced angina (chest pain) during exercise.
    - **1**: Yes
    - **0**: No

10. ST Depression Induced by Exercise Relative to Rest (oldpeak)

- **Type**: Continuous (Numerical)
- **Description**: The depression in the ST segment of the electrocardiogram caused by exercise relative to rest. A higher value indicates a greater likelihood of heart disease.

11. Slope of the Peak Exercise ST Segment (slope)

- **Type**: Categorical
- **Description**: The slope of the peak exercise ST segment, which provides insights into the heart's condition during physical exertion.
    - **Value 1**: Upsloping
    - **Value 2**: Flat
    - **Value 3**: Downsloping

12. Number of Major Vessels Colored by Fluoroscopy (ca)

- **Type**: Categorical (Numerical)
- **Description**: The number of major vessels that are colored by fluoroscopy, a diagnostic test used to detect blockages in the coronary arteries. A higher number may indicate more severe heart disease.

13. Thalassemia (thal)

- **Type**: Categorical
- **Description**: A blood disorder that is often related to cardiovascular health. This feature indicates the presence of thalassemia, which may affect heart health.
    - **Value 3**: Normal
    - **Value 6**: Fixed defect
    - **Value 7**: Reversible defect

14. Heart Disease Diagnosis (target)

- **Type**: Categorical (Binary)
- **Description**: This is the target variable that indicates whether the individual has heart disease or not. It is the outcome that the model is predicting.
    - **0**: No heart disease
    - **1**: Heart disease present

In this project the input is cardiac disease dataset which is supervised dataset it includes labels of prediction result.

4.4 Algorithm and Modeling

Random Forest Algorithm

Random Forest is an ensemble learning method that operates by constructing multiple decision trees during training and outputting the mode of the classes (for classification) or the mean prediction (for regression) of the individual trees. It is widely used for its high accuracy, robustness to overfitting, and ability to handle large datasets with high dimensionality.

Concepts of Random Forest

1. **Decision Trees**: A decision tree is a flowchart-like structure where each internal node represents a decision based on a feature, each branch represents the outcome of the decision, and each leaf node represents a class label or a continuous value. Decision trees are prone to overfitting, especially with complex datasets.
2. **Ensemble Learning**: Random Forest addresses the limitations of individual decision trees by combining multiple trees. This reduces overfitting and improves generalization.
3. **Bootstrap Aggregating (Bagging)**: Random Forest uses bagging, a technique where each tree is trained on a random subset of the training data (sampled with replacement). This introduces diversity among the trees, making the model more robust.
4. **Feature Randomness**: When splitting a node during the construction of a tree, the algorithm considers only a random subset of features. This further increases diversity among the trees and reduces correlation between them.

How Random Forest Works

1. **Data Sampling**: From the training dataset, multiple subsets are created using bootstrapping (random sampling with replacement).
2. **Tree Construction**: For each subset, a decision tree is constructed. At each node, the best split is determined by considering only a random subset of features.
3. **Prediction**: For classification tasks, each tree votes for a class, and the class with the majority votes is selected as the final prediction. For regression tasks, the average prediction of all trees is taken.

4. **Aggregation**: The predictions of all trees are aggregated to produce the final output.

Advantages of Random Forest

- High accuracy and robustness.
- Handles missing values and outliers effectively.
- Reduces overfitting compared to individual decision trees.
- Provides feature importance scores, which help in understanding the contribution of each feature to the prediction.

How Random Forest is Applied in This Project

In this project, the Random Forest algorithm is used to predict the risk of heart disease based on a set of input features such as age, gender, blood pressure, cholesterol levels, and other clinical parameters. Below is a step-by-step explanation of how the algorithm is applied:

Step 1: Data Collection

- The system collects health-related data from users through a Django - based web interface. The data includes both numerical (e.g., age, blood pressure) and categorical (e.g., gender, smoking status) features.

Step 2: Data Preprocessing

- The collected data is preprocessed to ensure it is suitable for the Random Forest model. This includes
    - Handling missing values (e.g., imputation or removal).
    - Normalizing numerical features to a common scale.
    - Encoding categorical features into numerical values (e.g., one-hot encoding).
    - Splitting the data into training and testing sets (e.g., 80% training, 20% testing).

Step 3: Model Training

- The Random Forest model is trained on the preprocessed training data. During training:
    - Multiple decision trees are constructed using bootstrapped subsets of the training data.

- At each node of a tree, the best split is determined by evaluating a random subset of features.
- The process continues until the trees reach a specified depth or a minimum number of samples per leaf.

Step 4: Model Evaluation

- The trained model is evaluated on the testing dataset to assess its performance. Common evaluation metrics for classification tasks include
    - **Accuracy**: The proportion of correctly predicted instances.
    - **Precision**: The proportion of true positive predictions among all positive predictions.
    - **Recall**: The proportion of true positives identified correctly.
    - **F1-Score**: The harmonic mean of precision and recall.
    - **Confusion Matrix**: A table showing true positives, true negatives, false positives, and false negatives.

Step 5: Prediction

- Once the model is trained and evaluated, it is used to predict the risk of heart disease for new, unseen data. The input features are passed through the ensemble of decision trees, and the final prediction is determined by majority voting.

Step 6: Feature Importance

- The Random Forest algorithm provides a feature importance score, which indicates the contribution of each feature to the prediction. This helps in identifying the most significant risk factors for heart disease.

4.5 Training and Validation

Classification Using Random Forest Algorithm

Random forests (RF) are combination of tree predictors using decision tree such that each tree depends on the values of a random vector sampled independently and with the same distribution for all trees in the forest. The generalization error of a forest of tree classifiers depends on the strength of the individual trees in the forest and the correlation between them. They are more robust with respect to noise. It is a supervised classification algorithm used for the

prediction and it is considered as the superior due to its large number of trees in the forest giving improved accuracy than decision trees. Typically, the trees are trained independently and the predictions of the trees are combined through averaging. Random forest algorithm can use both for classification and the regression based on the problem domain. The algorithm for random forest is given below:

Step 1: Randomly select k features from entire m features, where $k << m$.
Step 2: Surrounded by the k features, calculate the node "d" using the best split point.
Step 3: Split the node into daughter nodes using the best split.
Step 4: Repeat 1 to 3 steps until l number of nodes has been reached.
Step 5: Construct forest by repeating steps 1 to 4 for n number times to create n number of trees.

Firstly, the k features are taken out of total m features. In the next stage, in each tree randomly select k features in order to find the root node by using the best split approach. The next stage involves calculating the daughter nodes using the same best split approach for the cardiac disease dataset. Similarly, the tree is formed from the root node and until all the leaf nodes are generated from the attributes. This randomly created tree forms the random forest that is used for making cardiac disease prediction in patients.

5. SYSTEM TESTING AND IMPLEMENTATION

5.1. System Testing

Testing is the analysis of source or executable code and the controlled execution of the executable code to identify defects that compromise a Java program's integrity. Such defects often result in erratic behavior or the premature termination of the program during execution.

The Software testing process commences once the program is created and the documentation and related data structures are designed. Software testing essential for correcting errors . Otherwise the program or the project is said to be not complete. Software testing is a process of checking whether the developed system is working according to the original objectives and requirements. The system should be tested experimentally with test data so as to ensure that the system works according to the required specification. Software testing is a critical element of software quality assurance and represents the ultimate review of specification, design and coding. After the coding phase, computer programs are available that can be executed for testing purposes. This implies that testing not only has to uncover errors introduced during coding, but also errors introduced during the previous phases.

Testing objectives

The testing objectives are summarized in the following three steps

- Testing is a process of executing a program with the intent of finding an error.
- A good test case is one that has high probability of finding an as-yet undiscovered error.
- A successful test is one that uncovers an as yet undiscovered error.

Testing principles

- All the tests should be traceable to customer requirement.
- Tests should be planned long before testing begins that is the test planning can bring as soon as the requirement model is complete.

The following are the attributes of the good test:

A good test has high probability of finding an error.

- A good test is not redundant.
- A good test should be “best of breed”.
- A good test should be neither too simple nor too complex.

Testing methodologies

System testing is stage of implementation which is aimed at ensuring that the system works accurately and efficient before live operation commences. Testing is vital to the success of the system. System testing makes a logical assumption that if all the parts of the system are correct, the goal will be successfully achieved.

Types of Testing

The testing steps are

- Unit testing
- Integration testing
- Validation testing
- Performance testing

Unit Testing

Unit testing focuses verification efforts on the smallest unit of software design, the module. This is also known as "Module Testing". The modules are tested separately. This testing is carried out during programming stage itself. Unit testing specifies paths in the module's control structure to ensure complete coverage and maximum error detection. This test focuses on each module individually, ensuring that it functions properly as a unit.

Integration Testing

Data can be lost across the interface, one module can have an adverse effect on others. Integration testing is a systematic testing for constructing program structure. While at the same time conducting tests to uncover errors associated within the interface. Integration testing addresses the issues associated with the dual problems of verification and program construction. After the software has been integrated a set of high order sets are conducted. The objective is to take unit tested modules and combine them test it as a whole. Thus, in the integration-testing step, all the errors uncovered are corrected for the next testing steps.

Validation Testing

The outputs that come out of the system are as a result of the inputs that go in to the system. So, for the correct and the expected outputs the inputs that go in to the system should be

correct and proper. So this testing is done to check if the inputs are correct and they are validated before it goes in to the system for processing.

Performance Testing

Performance testing is designed to test the run-time performance of software within the context of an integrated system. It requires both hardware and software instrumentation. It is often necessary to measure resource utilization in an exacting fashion.

5.2. System Implementation

System implementation is the stage of the project that the theoretical design is turned into a working system. If the implementation stage is not properly planned and controlled, it can cause error. Thus it can be considered to be the most crucial stage in achieving a successful new system and in giving the user confidence that the new system will work and be effective.

Normally this stage involves setting up a coordinating committee, which will act as a sounding board for ideas; complaints and problem. The first task is implementation planning; i.e., deciding on the methods and time scale to be adopted. Apart from planning two major task of preparing for implementation are, education takes place much earlier in the project; at the implementation stage the emphasis must be on training in new skills to give staff confidence they can use the system. Once staff has been trained, the system can be tested.

After the implementation phase is completed and the user staff is adjusted to the changes created by the candidate system, evaluation and maintenance is to bring the new system to standards. The activities of the implementation phase can be summarized as,

- Implementation planning
- Education planning
- System planning

Implementation Steps

The predict of cardiac disease is done in three phases: feature selection process in this process we will automatically or manually select those features which contribute most to your prediction variable or output in which you are interested in. The second phase is applying the Random Forest machine learning algorithm in which the data will be trained and tested. The third and the last phase is the User interface in which the user will enter his details and then the

machine learning models will predict that the user will have cardiac diseases in future or not.

Step 1

Importing Libraries

```
import NumPy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
from sklearn.model_selection import KFold, StratifiedKFold, cross_val_score
from sklearn import linear_model, tree, ensemble
```

Step 2

Reading the Data from CSV file

```
df = pd.read_csv("dataset.csv", delimiter=",")
df.info()
data = df.to_numpy()
print(data.shape)
```

Step 3

Train-Test Split

X_train, X_test,y_train, y_test=train_test_split(X,y,test_size=0.25,random_state=40)

We split the whole dataset into trainset and testset which contains 75% train and 25% test.We can include this train set into classifiers to train our model and the test set is useful for predicting the performance of the model by different classifiers.

Step 4

Classification Using Random Forest Algorithm

Random forests (RF) are combination of tree predictors using decision tree such that each tree depends on the values of a random vector sampled independently and with the same distribution for all trees in the forest. The generalization error of a forest of tree classifiers depends on the strength of the individual trees in the forest and the correlation between them.

They are more robust with respect to noise. It is a supervised classification algorithm used for the prediction and it is considered as the superior due to its large number of trees in the forest giving improved accuracy than decision trees. Typically, the trees are trained independently and the predictions of the trees are combined through averaging. Random forest algorithm can use both for classification and the regression based on the problem domain. The algorithm for random forest is given below:

Step 1: Randomly select k features from entire m features, where $k << m$.
Step 2: Surrounded by the *k* features, calculate the node "*d*" using the best split point.
Step 3: Split the node into daughter nodes using the best split.
Step 4: Repeat 1 to 3 steps until l number of nodes has been reached.
Step 5: Construct forest by repeating steps 1 to 4 for n number times to create n number of trees.

Firstly, the *k* features are taken out of total *m* features. In the next stage, in each tree randomly select *k* features in order to find the root node by using the best split approach. The next stage involves calculating the daughter nodes using the same best split approach for the cardiac disease dataset. Similarly, the tree is formed from the root node and until all the leaf nodes are generated from the attributes. This randomly created tree forms the random forest that is used for making cardiac disease prediction in patients.

Step 5

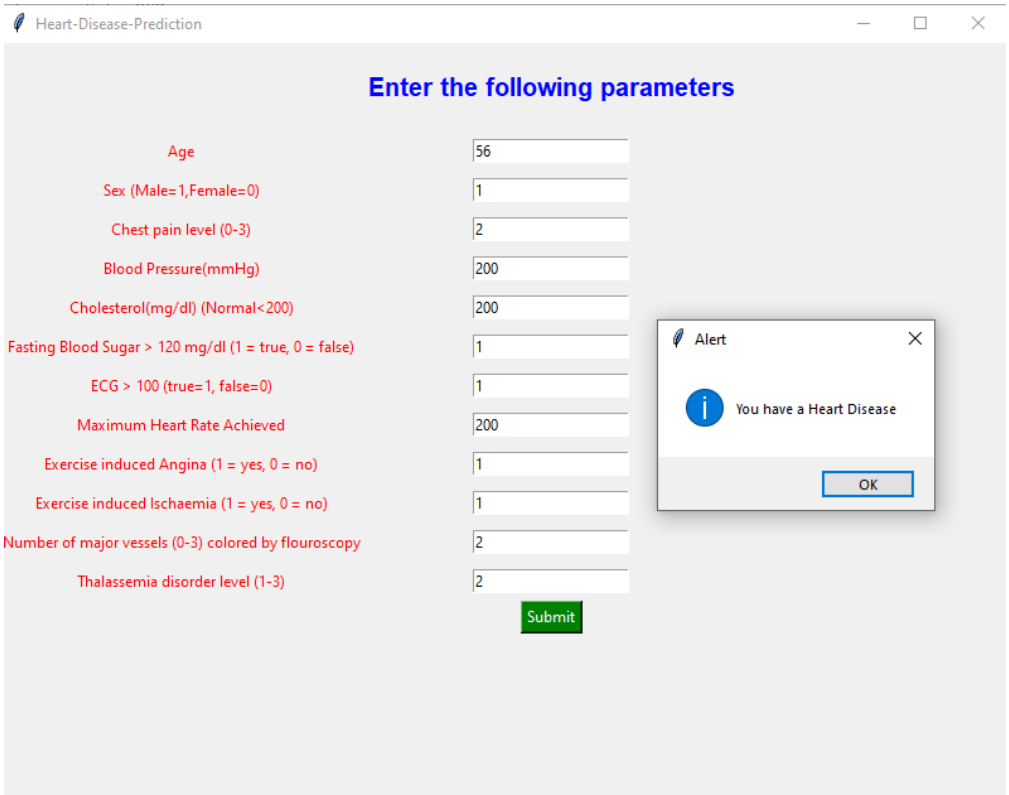
Creating GUI Application using Python-Tkinter

```
import pandas as pd
import numpy as np
from tkinter import *
from random_forest import RandomForest
def mod():
defheart_disease(new_input):
df = pd.read_csv("dataset.csv", delimiter=",")
    data = df.to_numpy()
n_samples, n_features = data.shape
n_features -= 1
    X = data[:,0:n_features]
```

```
        y = data[:,n_features]
clf = RandomForest(n_trees=20, max_depth=10)
clf.fit(X, y)
y_pred = clf.predict(new_input)
        return y_pred
    window = Tk()
window.title("Heart-Disease-Prediction")
window.geometry('550x400')
lbl = Label(window, text="Enter the following parameters", font=("Arial Bold", 10))
lbl.grid(column=2, row=0)
lbl = Label(window, text="Age")
lbl.grid(column=1, row=1)
    age = Entry(window,width=20)
age.grid(column=2, row=1)
age.focus()
lbl = Label(window, text="Sex (Male=1,Female=0)")
lbl.grid(column=1, row=2)
    sex = Entry(window,width=20)
sex.grid(column=2, row=2)
sex.focus()
lbl = Label(window, text="Chest pain level (0-3)")
lbl.grid(column=1, row=3)
cp = Entry(window,width=20)
cp.grid(column=2, row=3)
cp.focus()
lbl = Label(window, text="Blood Pressure(mmHg)")
lbl.grid(column=1, row=4)
    bps = Entry(window,width=20)
bps.grid(column=2, row=4)
bps.focus()
lbl = Label(window, text="Cholesterol(mg/dl) (Normal<200)")
```

```
lbl.grid(column=1, row=5)
chol = Entry(window,width=20)
chol.grid(column=2, row=5)
chol.focus()
lbl = Label(window, text="Fasting Blood Sugar > 120 mg/dl (1 = true, 0 = false)")
lbl.grid(column=1, row=6)
fbs = Entry(window,width=20)
fbs.grid(column=2, row=6)
fbs.focus()
lbl = Label(window, text="ECG > 100 (true=1, false=0)")
lbl.grid(column=1, row=7)
ecg = Entry(window,width=20)
ecg.grid(column=2, row=7)
ecg.focus()
lbl = Label(window, text="Maximum CardiacRate Achieved")
lbl.grid(column=1, row=8)
mhr = Entry(window,width=20)
mhr.grid(column=2, row=8)
mhr.focus()
lbl = Label(window, text="Exercise induced Angina (1 = yes, 0 = no)")
lbl.grid(column=1, row=9)
eia = Entry(window, width=20)
eia.grid(column=2, row=9)
eia.focus()
lbl = Label(window, text="Exercise induced Ischaemia (1 = yes, 0 = no)")
lbl.grid(column=1, row=11)
    slope = Entry(window, width=20)
slope.grid(column=2, row=11)
slope.focus()
lbl = Label(window, text="Number of major vessels (0-3) colored by flouroscopy")
lbl.grid(column=1, row=12)
```

```
ca = Entry(window, width=20)
ca.grid(column=2, row=12)
ca.focus()
lbl = Label(window, text="Thalassemia disorder level (1-3)")
lbl.grid(column=1, row=13)
thal = Entry(window, width=20)
thal.grid(column=2, row=13)
thal.focus()
def clicked():
new_input = list(map(int, [age.get(), sex.get(), cp.get(), bps.get(), chol.get(), fbs.get(), ecg.get(),
mhr.get(), eia.get(), slope.get(), ca.get(), thal.get()]))
new_input = np.reshape(new_input, (1, -1))
    output = heart_disease(new_input)
    if (output == 1):
        s = 'You have a CardiacDisease'
    else:
        s = "You are in Perfect Health"
ot.configure(text="Result: " + s)
btn = Button(window, text="Submit", command=clicked)
btn.grid(column=2, row=18)
ot = Label(window, text="", font=("Arial Bold", 10))
ot.grid(column=1, row=20)
window.mainloop()
mod()
```


Heart-Disease-Prediction
Enter the following parameters
Age
56
Sex (Male=1,Female=0)
1
Chest pain level (0-3)
2
Blood Pressure(mmHg)
200
Cholesterol(mg/dl) (Normal<200)
200
Fasting Blood Sugar > 120 mg/dl (1 = true, 0 = false)
1
ECG > 100 (true=1, false=0)
1
Maximum Heart Rate Achieved
200
Exercise induced Angina (1 = yes, 0 = no)
1
Exercise induced Ischaemia (1 = yes, 0 = no)
1
Number of major vessels (0-3) colored by flouroscopy
2
Thalassemia disorder level (1-3)
2
Submit
Alert
You have a Heart Disease
OK

CONCLUSION AND FUTURE ENHANCEMENTS

Conclusion

In conclusion, this project, AI Based Early Detection and Prevention of Cardiac Diseases is highly useful in daily life and, more importantly, in the healthcare sector. This system enables healthcare professionals to predict diseases based on patients' general information and symptoms.

If the health industry adopts this project, it can significantly reduce the workload of doctors by assisting them in accurately predicting a patient's condition. This enhances efficiency and improves patient care.

The early prognosis of cardiac diseases can aid in making decisions on lifestyle changes in high risk patients and in turn reduce the complications, which can be a great milestone in the field of medicine. This project resolved the feature selection i.e. backward elimination and successfully predicts the cardiac disease, with 85% accuracy. The model used was Random Forest Algorithm. Further for its enhancement, we can train on models and predict the types of cardiovascular diseases providing recommendations to the users, and also use more enhanced models.

Future Enhancements

To address these issues, we can create another classification that operates on the timeline itself, selecting tree splits based on intervals in the past. By modifying standard variable importance techniques for random forests, we are also able to present a system data of how the importance of a given variable varies over time, improving interpretability of the model. However, they don't fully take into account the temporal nature of system data. Also, while individual trees are relatively human-interpretable, the weighted combination of dozens or hundreds of trees is not. The problem is exacerbated by the fact that the trees in random forest are typically not pruned, often resulting in trees with larger depth than those learned by decision tree algorithms like ID3 and C4.5.

- Facility for modifying user details.
- More interactive user interface.
- Facilities for backup creation.
- Can be done as a webpage.
- Can be done as a mobile application.

BIBLIOGRAPHY

Reference books:

1. Lutz, M. (2013). Learning Python, 5th Edition (5 edition). Beijing: O'Reilly Media.
2. Tibbits, S., van der Harten, A., & Baer, S. (2011). Rhino Python Primer (3rd ed.).
3. Downey, A. B. (2015). "Think Python: How to Think Like a Computer Scientist (2edition)". Sebastopol, CA: O'Reilly Media.
4. Greg Wilson. "Data crunching: solve everyday problems using Java, Python and more. The pragmatic programmers", Pragmatic Bookshelf, Raleigh.
5. Guido van Rossum and Fred L. Drake, Jr. "The Python Tutorial — An Introduction to Python". Network Theory Ltd., Bristol,
6. Michael Dawson. "Python programming for the absolute beginner". Premier Press Inc., Boston, MA, USA, 2003.
7. Harvey M. Deitel, Paul Deitel, Jonathan Liperi, and Ben Wiedermann "Python How To Program". P T R Prentice-Hall, Englewood Cliffs.
8. Brad Dayley. "Python phrasebook: essential code and commands. Developer's library". SAMS Publishing, Indianapolis,
9. Liza Daly. "Next-generation web frameworks in Python". O'Reilly & Associates, Inc., 103a Morris Street, Sebastopol.
10. Mike Dawson. "Python programming for the absolute beginner". Thomson Course Technology, Boston,
11. Peter Norton, Alex Samuel, David Aitel, Eric Foster-Johnson, Leonard Richardson, Jason Diamond, Aleatha Parker, Michael Roberts, "Begining Python", 2005.

Website References

1. https://www.python.org/
2. https://www.learnpython.org/
3. https://www.tutorialspoint.com/python/i
4. https://www.codecademy.com/learn/learn-python

APPENDICES

A. Sample Coding

```
import pandas as pd
import numpy as np
from tkinter import *
from tkinter import messagebox
from random_forest import RandomForest
def mod():
defheart_disease(new_input):
df = pd.read_csv("dataset.csv", delimiter=",")
        data = df.to_numpy()
n_samples, n_features = data.shape
n_features -= 1
        X = data[:,0:n_features]
        y = data[:,n_features]
clf = RandomForest(n_trees=20, max_depth=10)
clf.fit(X, y)
y_pred = clf.predict(new_input)
        return y_pred
    window = Tk()
window.title("Heart-Disease-Prediction")
window.geometry('800x600')
lbl = Label(window, text="Enter the following parameters",fg='blue', font=("Arial Bold", 15))
lbl.grid(column=2, row=0,pady=20)
lbl = Label(window, text="Age",fg='red')
lbl.grid(column=1, row=1, pady=5)
    age = Entry(window,width=20)
age.grid(column=2, row=1,pady=5)
age.focus()
lbl = Label(window, text="Sex (Male=1,Female=0)",fg='red')
lbl.grid(column=1, row=2, pady=5)
```

```
sex = Entry(window,width=20)
sex.grid(column=2, row=2, pady=5)
sex.focus()
lbl = Label(window, text="Chest pain level (0-3)",fg='red')
lbl.grid(column=1, row=3, pady=5)
cp = Entry(window,width=20)
cp.grid(column=2, row=3, pady=5)
cp.focus()
lbl = Label(window, text="Blood Pressure(mmHg)",fg='red')
lbl.grid(column=1, row=4, pady=5)
    bps = Entry(window,width=20)
bps.grid(column=2, row=4, pady=5)
bps.focus()
lbl = Label(window, text="Cholesterol(mg/dl) (Normal<200)",fg='red')
lbl.grid(column=1, row=5, pady=5)
chol = Entry(window,width=20)
chol.grid(column=2, row=5, pady=5)
chol.focus()
lbl = Label(window, text="Fasting Blood Sugar > 120 mg/dl (1 = true, 0 = false)",fg='red')
lbl.grid(column=1, row=6, pady=5)
fbs = Entry(window,width=20)
fbs.grid(column=2, row=6, pady=5)
fbs.focus()
lbl = Label(window, text="ECG > 100 (true=1, false=0)",fg='red')
lbl.grid(column=1, row=7, pady=5)
ecg = Entry(window,width=20)
ecg.grid(column=2, row=7, pady=5)
ecg.focus()
lbl = Label(window, text="Maximum CardiacRate Achieved",fg='red')
lbl.grid(column=1, row=8, pady=5)
mhr = Entry(window,width=20)
```

```
mhr.grid(column=2, row=8, pady=5)
mhr.focus()
lbl = Label(window, text="Exercise induced Angina (1 = yes, 0 = no)",fg='red')
lbl.grid(column=1, row=9, pady=5)
eia = Entry(window, width=20)
eia.grid(column=2, row=9, pady=5)
eia.focus()
lbl = Label(window, text="Exercise induced Ischaemia (1 = yes, 0 = no)",fg='red')
lbl.grid(column=1, row=11, pady=5)
    slope = Entry(window, width=20)
slope.grid(column=2, row=11, pady=5)
slope.focus()
lbl = Label(window, text="Number of major vessels (0-3) colored by flouroscopy",fg='red')
lbl.grid(column=1, row=12, pady=5)
ca = Entry(window, width=20)
ca.grid(column=2, row=12, pady=5)
ca.focus()
lbl = Label(window, text="Thalassemia disorder level (1-3)",fg='red')
lbl.grid(column=1, row=13, pady=5)
thal = Entry(window, width=20)
thal.grid(column=2, row=13, pady=5)
thal.focus()
def clicked():
new_input = list(map(int, [age.get(), sex.get(), cp.get(), bps.get(), chol.get(), fbs.get(), ecg.get(),
mhr.get(), eia.get(), slope.get(), ca.get(), thal.get()]))
new_input = np.reshape(new_input, (1, -1))
        output = heart_disease(new_input)
        if (output == 1):
            s = 'You have a CardiacDisease'
messagebox.showinfo('Alert',s)
        else:
```

```
        s = "You are in Perfect Health"
messagebox.showinfo('Congrats',s)
btn = Button(window, text="Submit",fg='white',bg='green', command=clicked)
btn.grid(column=2, row=18)
ot = Label(window, text="", font=("Arial Bold", 10),fg='blue')
ot.grid(column=2, row=25, pady=10)
window.mainloop()
mod()
from collections import Counter
import numpy as np
def entropy(y):
n_occurances = np.bincount(y)
ps = n_occurances / len(y)
    return -np.sum([p * np.log2(p) for p in ps if p > 0])
class Node:
def __init__(
        self, feature=None, threshold=None, left=None, right=None, *, value=None
    ):
self.feature = feature
self.threshold = threshold
self.left = left
self.right = right
self.value = value
defis_leaf_node(self):
        return self.value is not None
class DecisionTree:
def __init__(self, min_samples_split=2, max_depth=100, n_feats=10):
self.min_samples_split = min_samples_split
self.max_depth = max_depth
self.n_feats = n_feats
self.root = None
```

```
def fit(self, X, y):
self.n_feats = X.shape[1] if not self.n_feats else min(self.n_feats, X.shape[1])
self.root = self._grow_tree(X, y)
def predict(self, X):
    return np.array([self._traverse_tree(x, self.root) for x in X])
def _grow_tree(self, X, y, depth=0):
n_samples, n_features = X.shape
n_labels = len(np.unique(y))
    # stopping criteria
    if (
        depth >= self.max_depth
        or n_labels == 1
        or n_samples<self.min_samples_split
    ):
leaf_value = self._most_common_label(y)
        return Node(value=leaf_value)
feat_idxs = np.random.choice(n_features, self.n_feats, replace=False)
    # greedily select the best split according to information gain
best_feat, best_thresh = self._best_criteria(X, y, feat_idxs)
    # grow the children that result from the split
left_idxs, right_idxs = self._split(X[:,best_feat], best_thresh)
    left = self._grow_tree(X[left_idxs, :], y[left_idxs], depth + 1)
    right = self._grow_tree(X[right_idxs, :], y[right_idxs], depth + 1)
    return Node(best_feat, best_thresh, left, right)
def _best_criteria(self, X, y, feat_idxs):
best_gain = -1
split_idx, split_thresh = None, None
    for feat_idx in feat_idxs:
X_column = X[:,feat_idx]
        thresholds = np.unique(X_column)
        for threshold in thresholds:
```

```
            gain = self._information_gain(y, X_column, threshold)
            if gain >best_gain:
best_gain = gain
split_idx = feat_idx
split_thresh = threshold
        return split_idx, split_thresh
def _information_gain(self, y, X_column, split_thresh):
        # parent loss
parent_entropy = entropy(y)
        # generate split
left_idxs, right_idxs = self._split(X_column, split_thresh)
        if len(left_idxs) == 0 or len(right_idxs) == 0:
            return 0
        # weighted avg. of the loss for the children
        n = len(y)
n_l, n_r = len(left_idxs), len(right_idxs)
e_l, e_r = entropy(y[left_idxs]), entropy(y[right_idxs])
child_entropy = (n_l / n) * e_l + (n_r / n) * e_r
        # information gain is difference in loss before vs. after split
ig = parent_entropy - child_entropy
        return ig
def _split(self, X_column, split_thresh):
left_idxs = np.argwhere(X_column<= split_thresh).flatten()
right_idxs = np.argwhere(X_column>split_thresh).flatten()
        return left_idxs, right_idxs
def _traverse_tree(self, x, node):
        if node.is_leaf_node():
            return node.value
        if x[node.feature] <= node.threshold:
            return self._traverse_tree(x, node.left)
        return self._traverse_tree(x, node.right)
```

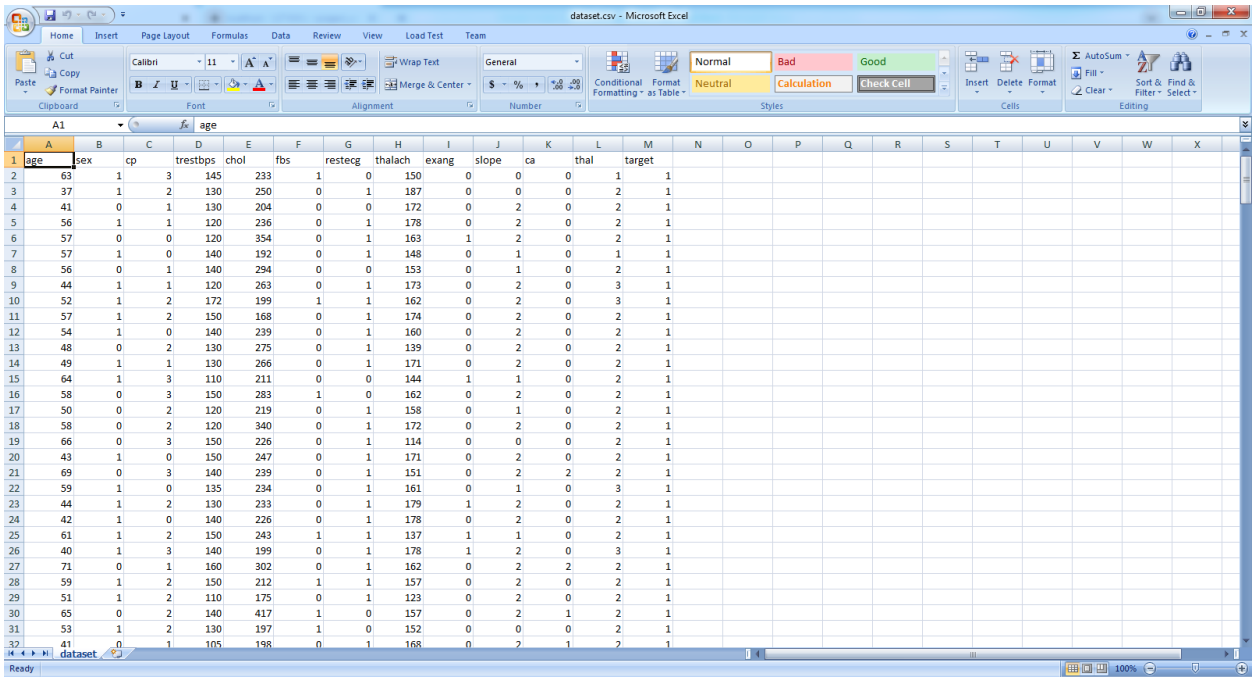
```
def _most_common_label(self, y):
        counter = Counter(y)
most_common = counter.most_common(1)[0][0]
        return most_common
from collections import Counter
import numpy as np
import pandas as pd
from decision_tree import DecisionTree
defbootstrap_sample(X, y):
n_samples = X.shape[0]
idxs = np.random.choice(n_samples, size=n_samples, replace=True)
    return X[idxs], y[idxs]
defmost_common_label(y):
    counter = Counter(y)
most_common = counter.most_common(1)[0][0]
    return most_common
class RandomForest:
def __init__(self, n_trees=100, min_samples_split=2, max_depth=100, n_feats=None):
self.n_trees = n_trees
self.min_samples_split = min_samples_split
self.max_depth = max_depth
self.n_feats = n_feats
self.trees = []
def fit(self, X, y):
self.trees = []
        for _ in range(self.n_trees):
            tree = DecisionTree(
min_samples_split=self.min_samples_split,
max_depth=self.max_depth,
n_feats=self.n_feats,
            )
```

```
X_samp, y_samp = bootstrap_sample(X, y)
tree.fit(X_samp, y_samp)
self.trees.append(tree)
def predict(self, X):
tree_preds = np.array([tree.predict(X) for tree in self.trees])
#[1111 0000 1111]
tree_preds = np.swapaxes(tree_preds, 0, 1)
#[101 101 101 101]
y_pred = [most_common_label(tree_pred) for tree_pred in tree_preds]
        return np.array(y_pred)
# Testing
if __name__ == "__main__":
    from sklearn.model_selection import train_test_split
def accuracy(y_true, y_pred):
        accuracy = np.sum(y_true == y_pred) / len(y_true)
        return accuracy
df = pd.read_csv("dataset.csv", delimiter=",")
df.info()
    data = df.to_numpy()
    print(data.shape)
n_samples, n_features = data.shape
n_features -= 1
    X = data[:,0:n_features]
    y = data[:,n_features]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=100)
clf = RandomForest(n_trees=50, max_depth=10)
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)
    print(np.concatenate((y_pred.reshape(len(y_pred),1), y_test.reshape(len(y_test),1)),1))
acc = accuracy(y_test, y_pred)
print("Accuracy in Random Forest Algorithm:", acc)
```

B. Sample Input

Sample Forms

Cardiac Disease- Data Set



age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	slope	ca	thal	target
63	1	3	145	233	1	0	150	0	0	0	1	1
37	1	2	130	250	0	1	187	0	0	0	2	1
41	0	1	130	204	0	0	172	0	2	0	2	1
56	1	1	120	236	0	1	178	0	2	0	2	1
57	0	0	120	354	0	1	163	1	2	0	2	1
57	1	0	140	192	0	1	148	0	1	0	1	1
56	0	1	140	294	0	0	153	0	1	0	2	1
44	1	1	120	263	0	1	173	0	2	0	3	1
52	1	2	172	199	1	1	162	0	2	0	3	1
57	1	2	150	168	0	1	174	0	2	0	2	1
54	1	0	140	239	0	1	160	0	2	0	2	1
48	0	2	130	275	0	1	139	0	2	0	2	1
49	1	1	130	266	0	1	171	0	2	0	2	1
64	1	3	110	211	0	0	144	1	1	0	2	1
58	0	3	150	283	1	0	162	0	2	0	2	1
50	0	2	120	219	0	1	158	0	1	0	2	1
58	0	2	120	340	0	1	172	0	2	0	2	1
66	0	3	150	226	0	1	114	0	0	0	2	1
43	1	0	150	247	0	1	171	0	2	0	2	1
69	0	3	140	239	0	1	151	0	2	2	2	1
59	1	0	135	234	0	1	161	0	1	0	3	1
44	1	2	130	233	0	1	179	1	2	0	2	1
42	1	0	140	226	0	1	178	0	2	0	2	1
61	1	2	150	243	1	1	137	1	1	0	2	1
40	1	3	140	199	0	1	178	1	2	0	3	1
71	0	1	160	302	0	1	162	0	2	2	2	1
59	1	2	150	212	1	1	157	0	2	0	2	1
51	1	2	110	175	0	1	123	0	2	0	2	1
65	0	2	140	417	1	0	157	0	2	1	2	1
53	1	2	130	197	1	0	152	0	0	0	2	1
41	0	1	105	198	0	1	168	0	2	1	2	1

Random Forest Algorithm

random_forest.py - C:\Users\admin\Desktop\HeartDiseasePrediction\random_forest.py (3.9.7)

File Edit Format Run Options Window Help

```python
from collections import Counter
import numpy as np
import pandas as pd

from decision_tree import DecisionTree

def bootstrap_sample(X, y):
    n_samples = X.shape[0]
    idxs = np.random.choice(n_samples, size=n_samples, replace=True)
    return X[idxs], y[idxs]


def most_common_label(y):
    counter = Counter(y)
    most_common = counter.most_common(1)[0][0]
    return most_common


class RandomForest:
    def __init__(self, n_trees=100, min_samples_split=2, max_depth=100, n_feats=None):
        self.n_trees = n_trees
        self.min_samples_split = min_samples_split
        self.max_depth = max_depth
        self.n_feats = n_feats
        self.trees = []

    def fit(self, X, y):
        self.trees = []
        for _ in range(self.n_trees):
            tree = DecisionTree(
                min_samples_split=self.min_samples_split,
                max_depth=self.max_depth,
                n_feats=self.n_feats,
            )
            X_samp, y_samp = bootstrap_sample(X, y)
            tree.fit(X_samp, y_samp)
            self.trees.append(tree)

    def predict(self, X):
        tree_preds = np.array([tree.predict(X) for tree in self.trees])
        #[1111 0000 1111]
        tree_preds = np.swapaxes(tree_preds, 0, 1)
        #[101 101 101 101]
        y_pred = [most_common_label(tree_pred) for tree_pred in tree_preds]
        return np.array(y_pred)


# Testing
if __name__ == "__main__":

    from sklearn.model_selection import train_test_split

    def accuracy(y_true, y_pred):
        accuracy = np.sum(y_true == y_pred) / len(y_true)
        return accuracy

    df = pd.read_csv("dataset.csv", delimiter=",")
```

Ln: 70 Col: 26

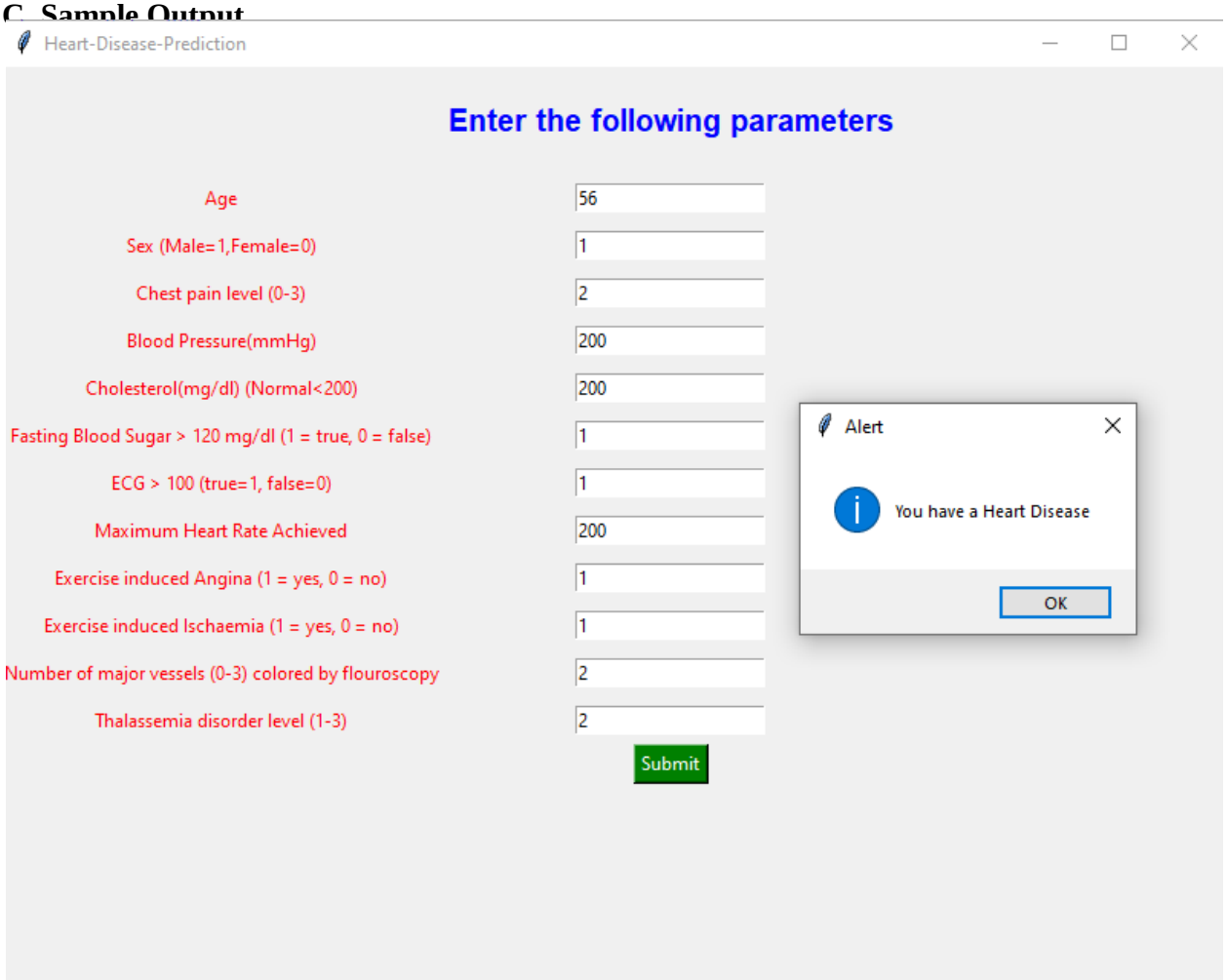
Result

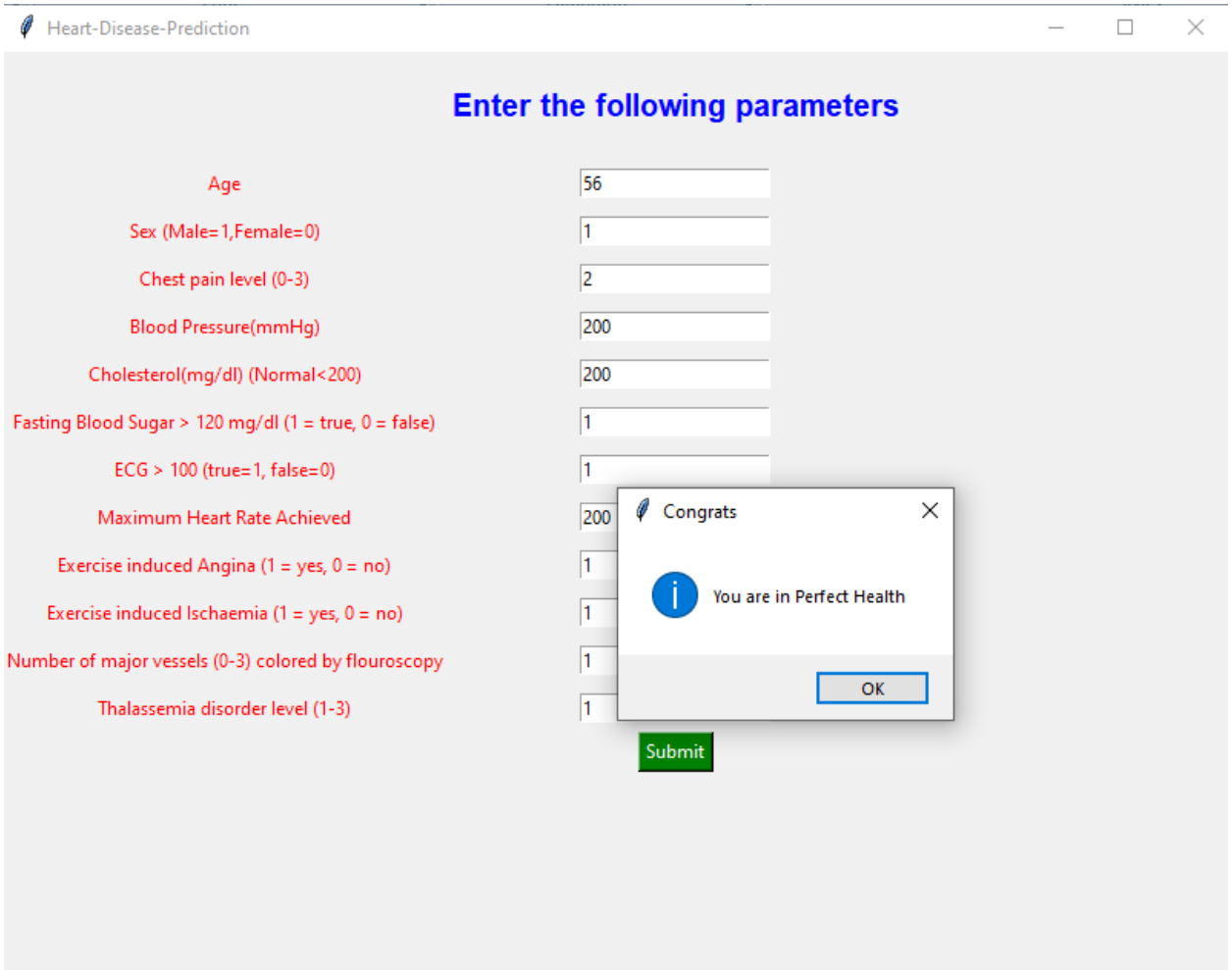
```
*IDLE Shell 3.9.7*
File  Edit  Shell  Debug  Options  Window  Help
Python 3.9.7 (tags/v3.9.7:1016ef3, Aug 30 2021, 20:19:38) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
==== RESTART: C:\Users\admin\Desktop\HeartDiseasePrediction\random_forest.py ===
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 303 entries, 0 to 302
Data columns (total 13 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   age       303 non-null    int64
 1   sex       303 non-null    int64
 2   cp        303 non-null    int64
 3   trestbps  303 non-null    int64
 4   chol      303 non-null    int64
 5   fbs       303 non-null    int64
 6   restecg   303 non-null    int64
 7   thalach   303 non-null    int64
 8   exang     303 non-null    int64
 9   slope     303 non-null    int64
 10  ca        303 non-null    int64
 11  thal      303 non-null    int64
 12  target    303 non-null    int64
dtypes: int64(13)
memory usage: 30.9 KB
(303, 13)
Ln: 5  Col: 0
```

Result (Accuracy)

```
IDLE Shell 3.9.7
File  Edit  Shell  Debug  Options  Window  Help
Python 3.9.7 (tags/v3.9.7:1016ef3, Aug 30 2021, 20:19:38) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
==== RESTART: C:\Users\admin\Desktop\HeartDiseasePrediction\random_forest.py ===
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 303 entries, 0 to 302
Data columns (total 13 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   age       303 non-null    int64
 1   sex       303 non-null    int64
 2   cp        303 non-null    int64
 3   trestbps  303 non-null    int64
 4   chol      303 non-null    int64
 5   fbs       303 non-null    int64
 6   restecg   303 non-null    int64
 7   thalach   303 non-null    int64
 8   exang     303 non-null    int64
 9   slope     303 non-null    int64
 10  ca        303 non-null    int64
 11  thal      303 non-null    int64
 12  target    303 non-null    int64
dtypes: int64(13)
memory usage: 30.9 KB
(303, 13)
Squeezed text (61 lines).
Accuracy in Random Forest Algorithm: 0.8524590163934426
>>>
Ln: 28  Col: 4
```

C. Sample Output




Heart-Disease-Prediction
Enter the following parameters
Age
56
Sex (Male=1,Female=0)
1
Chest pain level (0-3)
2
Blood Pressure(mmHg)
200
Cholesterol(mg/dl) (Normal<200)
200
Fasting Blood Sugar > 120 mg/dl (1 = true, 0 = false)
1
ECG > 100 (true=1, false=0)
1
Maximum Heart Rate Achieved
200
Exercise induced Angina (1 = yes, 0 = no)
1
Exercise induced Ischaemia (1 = yes, 0 = no)
1
Number of major vessels (0-3) colored by flouroscopy
1
Thalassemia disorder level (1-3)
1
Congrats
You are in Perfect Health
OK
Submit